

Перв. примен.	
---------------	--

УТВЕРЖДЕН
643.04832915.10190-01 33 01-
ЛУ

ИЗДЕЛИЕ Семейство ПЛК Logic Box
СИСТЕМНОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ
Руководство программиста
643.04832915.10190-01 33 01

Листов 66

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

АННОТАЦИЯ

Системное программное обеспечение семейства ПЛК Logic Vox включает встроенное ПО аппаратных компонентов (модулей) ПЛК и взаимодействующее с ним ПО управления, работающее на компьютере в среде ОС GNU/Linux. В настоящем руководстве программиста приведена информация, необходимая для настройки ПЛК/УСО Logic Vox на конкретное применение в составе систем промышленной автоматизации.

Настоящий документ подготовлен в соответствии с требованиями ГОСТ-19.504-79 [1]

СОДЕРЖАНИЕ

1. Назначение и условия применения ПО	7
1.1. Назначение и взаимодействие компонентов ПО Logic Box	8
1.1.1. Встроенное ПО базового модуля	8
1.1.2. Встроенное ПО модулей ввода-вывода	9
1.1.3. Конфигурационный файл conf.yml	9
1.1.4. Программа lbconf	9
1.1.5. Программа lbcmd	9
1.1.6. Программа lbdiscover	9
1.1.7. Программа lbota	10
1.1.8. Программа lblog	10
1.1.9. Программа lbupload	10
1.1.10. Интегрированная среда разработки 4diac IDE	10
1.2. Условия применения ПО	11
2. Характеристики программы	12
2.1. Принципы организации ПО Logic Box	12
2.1.1. Брокер и конфигурация	12
2.1.2. Прикладные программы на языке ST (IEC 61131-3)	13
2.1.3. Прикладные программы на языке FBD (IEC 61499)	14
2.1.4. Передача данных между ПЛК	14
2.1.5. Интерфейс управления и отладки	15
3. Обращение к программе	16
3.1. Программа discover	16
3.1.1. Опциональные аргументы программы lbdiscover	17
3.2. Программа lbconf	18

3.2.1. Опциональные аргументы программы lbconf	18
3.2.2. Замечания по реализации	18
3.3. Программа lbcmd	20
3.3.1. Команда «считывание переменных»	20
3.3.2. Команда «установка значений переменных»	20
3.3.3. Команда «установка значений переменных в режиме force» .	20
3.3.4. Команда «отмена режима force»	21
3.3.5. Системные переменные	21
3.3.6. Команда calibrate	22
3.3.7. Команда fboot	22
3.3.8. Команда getconf	22
3.3.9. Команда json	23
3.3.10. Команда nofboot	23
3.3.11. Команда restart	23
3.3.12. Команда settime	23
3.3.13. Команда stats	24
3.3.14. Опциональные аргументы программы lbcmd	24
3.4. Программа lbota	25
3.4.1. Формат обращения	25
3.4.2. Опциональные аргументы программы lbota	25
3.4.3. Примеры обращения	26
3.5. Программа lblog	26
3.5.1. Формат обращения	26
3.5.2. Опциональные аргументы программы lblog	27
3.5.3. Примеры обращения	27
3.6. Программа lbupload	29

3.6.1. Опциональные аргументы программы lbusload	29
3.6.2. Замечания по реализации	30
4. Входные и выходные данные	32
5. Конфигурация ПЛК Logic Box	33
5.1. Конфигурационный файл conf.yml	33
5.1.1. Многофайловая конфигурация	33
5.2. Разделы конфигурации	34
5.2.1. Раздел clock	34
5.2.2. Раздел forte	34
5.2.3. Раздел modbus_client	35
5.2.4. Раздел modbus_server	35
5.2.5. Раздел slot	36
5.2.6. Раздел var	36
5.2.7. Раздел wdt	37
5.3. Параметры конфигурации	38
5.3.1. Параметр direct (slot:iec104_server:io)	38
5.3.2. Параметр interval	38
5.3.3. Параметр module (slot)	39
5.3.4. Параметр macaddr (slot)	39
5.3.5. Параметр quality (slot:iec104_server:command)	39
5.3.6. Параметр quality (slot:iec104_server:meas)	40
5.3.7. Параметр quality (slot:iec104_server)	40
5.3.8. Параметр select (slot:iec104_server:io)	40
5.3.9. Параметр select_timeout (slot:iec104_server:io)	41
5.3.10. Параметр select_timeout (slot:iec104_server)	41
5.3.11. Параметр period (task)	41

5.3.12. Параметр period (в других контекстах)	42
5.4. Примеры конфигурации	42
6. Диагностика	49
7. Сообщения	62
Перечень использованных источников	64
Перечень сокращений	65

1. НАЗНАЧЕНИЕ И УСЛОВИЯ ПРИМЕНЕНИЯ ПО

Семейство модульных ПЛК Logic Box предназначено для создания распределенных систем управления (автоматизации технологического процесса) с использованием сетей Ethernet и RS-485. ПО Logic Box включает следующие программы:

1) встроенное ПО: каждый модуль ПЛК Logic Box работает под управлением встроенного ПО, название которого совпадает с артикулом модуля (например, «Встроенное ПО LB241BC»);

2) ПО управления: программы, работающие на управляющем компьютере, обеспечивающие настройку ПЛК Logic Box на конкретное применение и контроль работы системы управления.

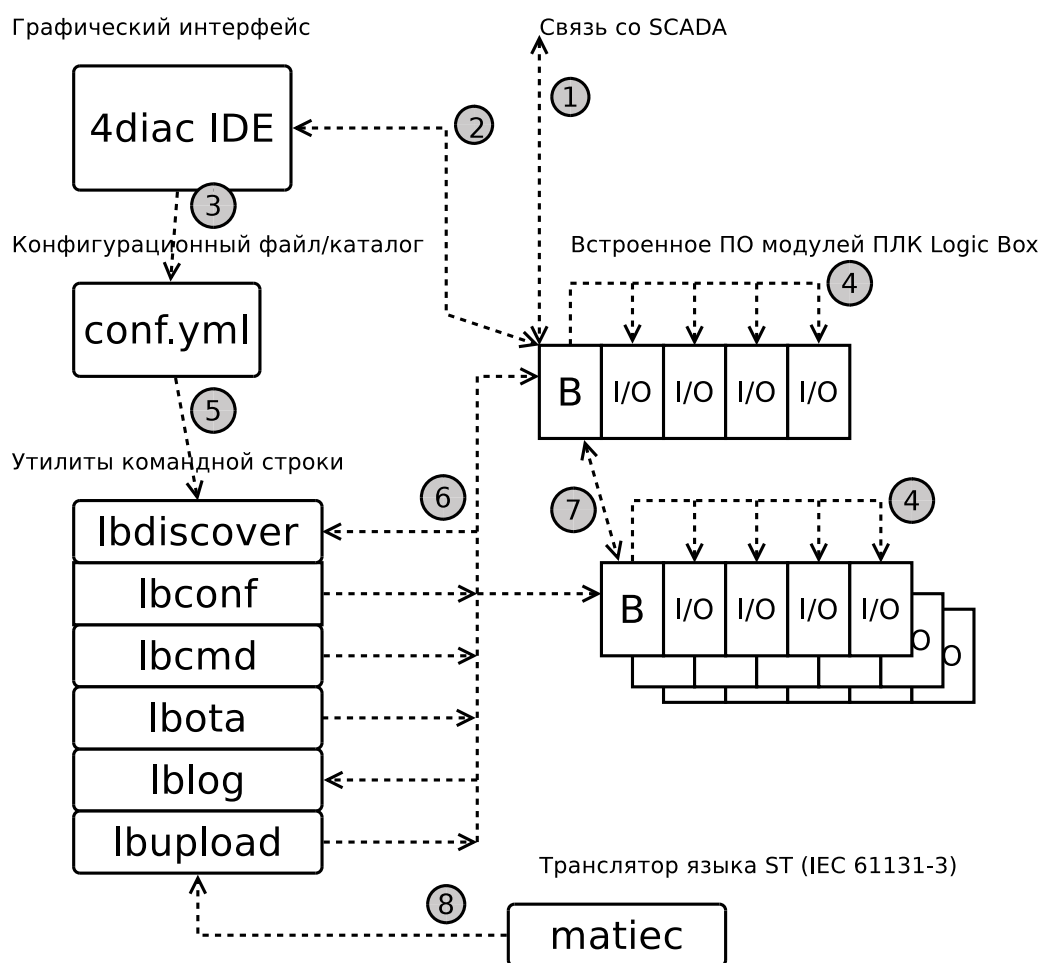


Рисунок 1. Информационные связи компонентов ПО Logic Box. Стрелки указывают основное направление передачи информации

1.1. Назначение и взаимодействие компонентов ПО Logic Box

1.1.1. Встроенное ПО базового модуля

Каждый ПЛК содержит как минимум один базовый модуль (В на рис. 1), например LB241BC или LB340CPU, который координирует работу модулей ввода-вывода (I/O). Встроенное ПО базового модуля получает команды управления от управляющего компьютера по протоколу NPC21_Modbus (стрелка 6 на рис. 1); в первую очередь это команда загрузки конфигурации из файла `conf.yml` утилитой `lbconf`. Конфигурация определяет режимы работы подсистем базового модуля и модулей ввода-вывода. Получив конфигурацию, ПО базового модуля загружает фрагменты конфигурации в модули ввода вывода, а затем в непрерывном режиме выполняет следующие функции:

- прием данных от модулей ввода-вывода по шине ПЛК (стрелка 4);

- прием и передача данных согласно загруженной конфигурации в SCADA (стрелка 1) и другие ПЛК (стрелка 7) по протоколам Modbus и IEC-104;

- передача данных в прикладные программы, выполняющиеся на ПЛК согласно конфигурации: программы на языке ST (IEC 61131-3) и программы на языке FBD (IEC 61499). Проекты IEC 61499 могут выполняются в ПЛК Logic Box одновременно с программами стандарта IEC 61131-3, взаимодействуя с последними через общее пространство именованных переменных;

- прием данных от прикладных программ;

- передача данных в модули ввода-вывода по шине ПЛК (стрелка 4) согласно конфигурации;

- передача данных на управляющий компьютер по запросу утилиты `lbcmd` в целях контроля работы ПЛК (отладочный интерфейс);

- прием и обработка запросов от управляющего компьютера по протоколу NPC21_Modbus (стрелка 6), генерируемых утилитами `lbconf`, `lbcmd` и т.д.;

- прием и обработка запросов по протоколу IEC 61499 от управляющего компьютера, генерируемых графическим интерфейсом `4diac IDE` (стрелка 2);

- прием и обработка данных по протоколу IEC 61499 от других ПЛК (стрелка 7).

1.1.2. Встроенное ПО модулей ввода-вывода

Встроенное ПО модулей ввода-вывода в непрерывном режиме транслирует данные между шиной ПЛК и портами ввода-вывода в соответствии с загруженной конфигурацией. В некоторых модулях (в частности, LB241AI4, LB340AO4) над данными выполняются нормирующие преобразования и применяются калибровочные поправки. Все модули поддерживают прием и обработку запросов протокола NPC21_Modbus от утилит lbconf, lbcmd, lblog; остальные утилиты не работают с модулями ввода-вывода, а только с базовыми модулями.

1.1.3. Конфигурационный файл conf.yml

Конфигурационный файл содержит описание параметров и режимов работы каждого модуля каждого ПЛК в распределенной системе управления в стандартном формате YAML [2]. Конфигурационный файл используется всеми утилитами (lbconf ... lbupload). Редактирование файла возможно в графическом интерфейсе 4diac IDE, или в любом текстовом редакторе, желательно имеющем поддержку синтаксиса YAML (например: emacs или vim).

1.1.4. Программа lbconf

Программа lbconf выбирает из файла conf.yml и загружает в указанный ПЛК относящийся к данному ПЛК фрагмент конфигурации, используя протокол NPC21_Modbus (стрелка 6).

1.1.5. Программа lbcmd

Программа lbcmd позволяет выполнять команды управления на ПЛК Logic Box, в частности устанавливать и считывать текущие значения переменных из памяти ПЛК в целях отладки.

1.1.6. Программа lbdiscover

Программа `lbdiscover` использует `multicast ping IPv6` для обнаружения устройств `Logic Box` в локальной сети `Ethernet`, в том числе устройств, ещё не имеющих конфигурации. Применяется при добавлении новых ПЛК в систему.

1.1.7. Программа `lbota`

Программа `lbota` выполняет удаленное (т.н. «over the air») обновление встроенного ПО указанного модуля ПЛК `Logic Box`.

1.1.8. Программа `lblog`

Программа `lblog` считывает содержимое журнала системных сообщений указанного модуля ПЛК `Logic Box` в целях расширенной диагностики.

1.1.9. Программа `lbupload`

Программа `lbupload` выполняет компиляцию программ на языке `ST IEC 61131-3` и загрузку исполняемого кода в ПЛК `Logic Box`. При компиляции программ программа `upload` использует транслятор `matiec` [3] (стрелка 8), программные средства «GNU Toolchain» из комплекта `SDK esp-idf` [4] версии 5.3.2 и утилиту `make` из используемого дистрибутива ОС `GNU/Linux`.

1.1.10. Интегрированная среда разработки `4diac IDE`

Среда `4diac IDE` [5] представляет собой свободное программное обеспечение (лицензия `Eclipse Public License v2.0`) для программирования распределенных систем промышленной автоматизации в соответствии со стандартом `IEC 61499` в графической форме диаграмм функциональных блоков (FBD). Среда `4diac IDE` взаимодействует по протоколу `TCP` (порт 61499) с компонентой `4diac FORTE`, встроенной в ПО базовых модулей ПЛК `Logic Box`, обеспечивающей загрузку и выполнение проектов `IEC 61499` в ПЛК `Logic Box`.

Экземпляры `4diac FORTE` в разных ПЛК, входящих в один распределенный проект `4diac`, взаимодействуют по протоколам `TCP/IP`; конкретный протокол

и порт определяются использованными в проекте коммуникационными функциональными блоками.

1.2. Условия применения ПО

Встроенное ПО Logic Box работает модулях ПЛК Logic Box соответствующих типов без дополнительных условий.

ПО управления работает в среде ОС Linux на компьютере с архитектурой AMD64, требует не менее 4ГБ оперативной памяти и не менее 10ГБ дискового пространства. При разработке и тестировании используется дистрибутив Debian GNU/Linux 12 (bookworm); возможна адаптация ПО управления под любой современный дистрибутив ОС Linux.

ПО управления взаимодействует с ПЛК Logic Box в локальной сети Ethernet по протоколу Modbus TCP. Для упрощения конфигурации ПЛК желательно наличие сервера DHCP в локальной сети, хотя возможно и статическое назначение адреса. Если в конфигурации ПЛК задан статический адрес IPv4, утилиты ПО управления используют Modbus TCP IPv4, иначе Modbus TCP IPv6.

2. ХАРАКТЕРИСТИКИ ПРОГРАММЫ

2.1. Принципы организации ПО Logic Box

ПЛК Logic Box позволяют одновременно (в режиме разделения времени) исполнять одну или несколько прикладных программ, написанных на языках стандарта IEC 61131-3 [6], реализующих логику управления. В текущей версии ПО поддерживается только один из четырех определенных в стандарте языков – язык ST.

Программы ПЛК поддерживают связь с объектом управления через входные и выходные переменные языка ST. Переменные соответствуют сигналам на интерфейсах модулей ПЛК, или данным удаленных устройств ввода-вывода, взаимодействующих с ПЛК по протоколам Modbus [7] или IEC 60870-5-104 [8].

2.1.1. Брокер и конфигурация

Центральным компонентом встроенного ПО ПЛК Logic Box является брокер данных, реализующий взаимодействие подсистем ПЛК по модели «издатель–подписчик». Под «подсистемами» имеются в виду как аппаратные компоненты (модули расширения, установленные в слотах шины ПЛК), так и программные, например `modbus_server`, `modbus_client`, `plcprog` (подсистема исполнения программ ПЛК). Когда одна подсистема (выступающая в этот момент как «издатель») вырабатывает новое значение переменной, брокер сохраняет обновленное значение для выдачи по запросам, а также пересылает новое значение подсистемам-подписчикам данной переменной. Каждая подсистема может быть издателем и подписчиком для произвольного числа переменных; конкретные связи «издатель-подписчики» устанавливаются (изменяются) в момент начальной (или повторной) загрузки конфигурации в ПЛК.

Конфигурация всех ПЛК, входящих в состав распределенной системы управления, описывается в формате YAML [2] в файле `conf.yml` и загружается в ПЛК командой `lbconf` (раздел 3.2). Конфигурация задает аппаратный состав ПЛК (установленные модули), режимы каналов ввода-вывода и настройку программных подсистем; директивы конфигурации описаны в разделе 5.

Описание конфигурации имеет иерархическую структуру, в которой каждой подсистеме и ее блокам (например, каждому каналу 4-канального модуля аналогового ввода) соответствует свой подраздел (поддерево иерархии). Для указания, что данный блок является «издателем» (источником) переменной *name*, в его подразделе указывается атрибут *var: name*. Для указания, что данный блок является «подписчиком» (получателем) переменной *name*, в его подразделе указывается атрибут *var_out: name*. У каждой переменной может быть только один издатель, и неограниченное число подписчиков. Исключением являются переменные с атрибутом *multisource*, у которых может быть несколько издателей; при использовании таких переменных прикладной программист должен обеспечить, чтобы интервал между изменением переменной разными издателями был не меньше времени, необходимого для распространения изменений всем подписчикам.

Область видимости переменных ограничивается одним ПЛК, т.е. одно имя переменной в разных ПЛК можно использовать для разных целей, фактически это разные переменные.

2.1.2. Прикладные программы на языке ST (IEC 61131-3)

Программы ПЛК на языке ST IEC 61131-3 создаются на управляющем компьютере с помощью текстового редактора и описываются в конфигурации (разделы *plcN: taskN: progN*): указывается имя файла с программой и параметры ее выполнения. Загрузка программ ПЛК реализована отдельно от загрузки конфигурации и выполняется программой *lbuupload* (раздел 3.6). Для запуска программы необходимо:

- 1) чтобы программа была загружена;
- 2) чтобы исполнение данной программы было разрешено в конфигурации (*plcN: taskN: progN: enable: y*);
- 3) чтобы был установлен общий флаг разрешения выполнения программ: *sys.run=1* (раздел 3.3.5).

Привязка программ ПЛК к физическим интерфейсам ПЛК осуществляется через переменные брокера. Как входные, так и выходные переменные должны

быть описаны в программе на языке ST как VAR_EXTERNAL, а также перечислены в разделе var конфигурации (см. 5.2.6); для входных переменных должен быть указан атрибут `init`.

Выходные переменные программ ПЛК обновляются часто (параметр `task:period`, раздел `refsec:conf.yml-task`), и поэтому обрабатываются брокером особо: если при обновлении не произошло изменения значения переменной, брокер не рассылает переменную подписчикам. Можно разрешить рассылку, задав для переменной атрибут `interval` в разделе var конфигурации (см. 5.2.6).

2.1.3. Прикладные программы на языке FBD (IEC 61499)

Программы ПЛК на языке FBD IEC 61499 (также называемые «проектами 4diac») разрабатываются в интегрированной среде 4diac IDE [5]. Среда позволяет загрузить («Развернуть», «Deploy») проект в указанный ПЛК и наблюдать за его работой (изменениями значений переменных во времени). Проект исполняется на ПЛК программной компонентой 4diac FORTE, входящей в состав встроенного ПО модулей LB241CPU и LB340CPU.

Привязка проектов 4diac к физическим интерфейсам ПЛК осуществляется через переменные брокера, перечисленные в разделе `forte` конфигурации (см. 5.2.2). В проектах 4diac переменные из списка `forte:var` необходимо привязать к функциональным блокам PUBLISH, а переменные `forte:var_out` к функциональным блокам SUBSCRIBE.

Для автоматической загрузки проекта forte на выполнение после перезагрузки ПЛК необходимо в 4diac IDE сгенерировать файл `forte.fboot` и загрузить его в постоянную память ПЛК (раздел 3.3.7).

2.1.4. Передача данных между ПЛК

Для передачи данных между ПЛК в распределенной системе управления используются протоколы Modbus или IEC-104; при этом на передающем ПЛК передаваемая переменная должна быть описана как `var_out` в подсистеме Modbus (или IEC-104), а на принимающем ПЛК должна быть соответствующая входная переменная, описанная как `var` в подсистеме Modbus (или IEC-104).

Имена переменных на передающем и принимающем ПЛК могут совпадать или различаться; соответствие между переменными устанавливается по адресам регистров Modbus (или объектов IEC-104).

2.1.5. Интерфейс управления и отладки

Для контроля и управления работой ПЛК реализован интерфейс управления и отладки, представленный для пользователя командой `lbcmd` (раздел 3.3). Интерфейс позволяет выводить значения переменных и устанавливать их, а также выполнять команды управления. Реализация базируется на передаче JSON-объектов с командами и данными по протоколу NPC21_Modbus (Modbus TCP или Modbus RTU, дополнительная функция `fn=107`) и допускает неограниченное расширение набора команд.

3. ОБРАЩЕНИЕ К ПРОГРАММЕ

3.1. Программа discover

Программа `lbdiscover` выполняет обнаружение ПЛК LogicBox 340 в локальной сети, непосредственно подключенной к компьютеру. Программа обнаруживает все ПЛК, как имеющие, так и не имеющие конфигурации. При запуске без аргументов программа `lbdiscover` выводит информацию о всех ПЛК LogicBox 340, присутствующих в локальной сети:

```
$ lbdiscover
plc1 LB241CPU [192.168.1.142] f4:12:fa:dd:c6:17 eth0
plc102 LB241CPU [192.168.1.70] f4:12:fa:dd:c5:e3 eth0
<noname> LB340CPU [192.168.2.139] 2c:bc:bb:11:36:73 eth1
```

Каждая строка в выдаче программы описывает один обнаруженный ПЛК:

`plc1`: имя ПЛК согласно конфигурации, загруженной в ПЛК на данный момент. Если в ПЛК еще не загружена конфигурация, вместо имени ПЛК выводится `<noname>`.

`192.168.1.226`: адрес IPv4, выданный ПЛК сервером DHCP или заданный статически при конфигурации ПЛК. Это сопутствующая информация, не имеющая большого значения: при обращениях к ПЛК по имени все утилиты определяют адрес автоматически по конфигурационному файлу. При изменении адреса ПЛК (выдачи нового адреса сервером DHCP или задания нового статического адреса в конфигурации) новый адрес автоматически регистрируется в системе mDNS. Если на компьютере настроено использование mDNS (в `/etc/nsswitch.conf`), можно обращаться к ПЛК не по IP-адресу, а по имени, например: `ping plc102.local`.

`f4:12:fa:dd:c5:e3`: адрес уровня 2 модели OSI (MAC-адрес Ethernet, EUI-48). Постоянный уникальный идентификатор данного ПЛК. Получение MAC-адресов ПЛК, не имеющих конфигурации, является главной целью программы `discover`. Адрес необходим для однозначной идентификации ПЛК при задании его конфигурации в файле `conf.yml`, он должен быть задан в обязательном параметре `macaddr: plcN: {macaddr: f4:12:fa:d4:d1:7f}`.

eth0: интерфейс локальной сети, в которой обнаружен ПЛК – на случай, когда управляющий компьютер имеет несколько интерфейсов локальной сети. Это сопутствующая информация, не имеющая большого значения: при обращениях к ПЛК по имени все утилиты определяют интерфейс автоматически по конфигурационному файлу.

3.1.1. Опциональные аргументы программы lbdiscover

--help: команда lbdiscover выдает краткую справку по использованию и завершается:

```
$ lbdiscover --help
Usage: lbdiscover [-n|--newonly] [-f /path/to/conf.yml]
```

-f /path/to/conf.yml задает альтернативный путь к файлу конфигурации. По умолчанию программа lbdiscover использует файл конфигурации *BASEDIR/conf/conf.yml*, который находится исходя из пути запуска програмы lbdiscover(*BASEDIR/bin/lbconf*);

-n: опция подавляет вывод информации о ПЛК, уже присутствующих в файле конфигурации, и удобна для обнаружения новых ПЛК, требующих конфигурации, при большом числе ПЛК в локальной сети.

3.2. Программа lbconf

Программа lbconf выполняет загрузку конфигурации ПЛК LogicBox 340. Конфигурация всех ПЛК, входящих в состав распределенной системы управления, задается в файле `conf.yml`. Формат файла конфигурации описан в разделе 5.

При запуске программы lbconf с указанием имени ПЛК в качестве аргумента программа выделяет из файла `conf.yml` раздел конфигурации, относящийся к данному ПЛК, сериализует в формате JSON и передает на ПЛК по протоколу Modbus TCP:

```
$ lbconf plc1  
Configuration status: OK
```

Между запуском программы lbconf и выдачей сообщения Configuration status: OK или сообщения об ошибке обычно проходит 2-5 секунд (не более 10).

3.2.1. Опциональные аргументы программы lbconf

`--help`: команда lbconf выдает краткую справку по использованию и завершается:

```
$ lbconf --help  
Usage: lbconf {PLCNAME|--all} [-f /path/to/conf.yml] [--bymac]
```

`-f /path/to/conf.yml` задает альтернативный путь к файлу конфигурации. По умолчанию программа lbconf использует файл конфигурации `BASEDIR/conf/conf.yml`, который находится исходя из пути запуска программы `lbconf(BASEDIR/bin/lbconf)`;

`--all` аргумент можно указать вместо имени ПЛК, если требуется обновить конфигурацию всех ПЛК, описанных в файле `conf.yml`.

`--bymac` устанавливает режим обращения к ПЛК по локальному (link local) адресу IPv6, даже если для целевого ПЛК в файле `conf.yml` задан статический адрес IPv4. См. раздел 3.2.2.

3.2.2. Замечания по реализации

Программа `lbconf` обращается к ПЛК по протоколу Modbus TCP с использованием дополнительной (user-defined) функции `fn=107`. По умолчанию обращение производится по локальному (link local, `fe80::/10`) адресу IPv6, сформированному на основе MAC-адреса, заданного обязательным параметром `macaddr` в конфигурационном файле `conf.yml`.

В случае, когда в конфигурационном файле `conf.yml` для целевого ПЛК задан статический адрес IPv4 (параметр `ipaddr`), обращение производится по этому адресу – это полезно в случае, если ПЛК находится не в одной локальной сети с управляющим компьютером. Можно потребовать использовать локальный адрес IPv6 с помощью параметра `--bymas` (раздел 3.2.1).

В ПЛК Logic Box сервер Modbus TCP IPv6 на стандартном порту 502 включен всегда; раздел `modbus_server` в файле `conf.yml` управляет только серверами Modbus TCP IPv4 и Modbus RTU. Таким образом, возможность обновления конфигурации ПЛК, находящегося в одной локальной сети с управляющим компьютером, сохраняется всегда, даже при неверных параметрах TCP/IP и при изменении параметра `modbus_server:tcp_port` в файле `conf.yml`.

3.3. Программа lbcmd

Программа lbcmd позволяет выполнять на ПЛК команды считывания и установки переменных в целях отладки, а также некоторые другие команды.

Программа lbcmd требует один обязательный аргумента: имя ПЛК из конфигурационного файла conf.yml, например plc1. Дополнительные аргументы варьируются в зависимости от выполняемой команды.

3.3.1. Команда «считывание переменных»

Следующая команда запрашивает у ПЛК с именем plc1 значения четырех переменных и выводит имена переменных и их значения на стандартный вывод:

```
$ lbcmd plc1 di0..2 cloop3  
2023-10-25 19:18:47 di0=0 di1=0 di2=1 cloop3=0.015
```

Программа lbcmd не имеет ограничения на количество переменных в запросе, но в текущей реализации встроенного ПО ПЛК есть ограничение на размер ответа ПЛК в формате JSON: 218 байтов. Таким образом, ограничение на количество переменных в запросе зависит от суммы длин имен переменных и их значений в текстовом виде.

3.3.2. Команда «установка значений переменных»

Следующая команда выполняет установку на ПЛК с именем plc1 значений двух переменных: переменная do0 получает значение 1, переменная с именем cloop1 получает значение 0.004. При отсутствии ошибок команда не выдает сообщений и завершается в кодом 0.

```
$ lbcmd plc1 do=1 cloop1=0.004
```

3.3.3. Команда «установка значений переменных в режиме force»

Приведенная ниже команда выполняет установку на ПЛК с именем plc1 значений двух переменных: переменная do0 получает значение 1, переменная с именем cloop1 получает значение 0.004. Ключ -F задает перевод переменных в «принудительный» («force») режим, в котором встроенное ПО ПЛК игнорирует

попытки изменения этих переменных со стороны программ ПЛК или модулей ввода-вывода. Режим «force» сохраняется до его отмены с помощью команды cmd -U, или до перезагрузки ПЛК.

При отсутствии ошибок команда не выдает сообщений и завершается в кодом 0.

```
$ lbcmd plc1 -F do0=1 cloop1=0.004
```

3.3.4. Команда «отмена режима force»

Следующая команда отменяет на ПЛК с именем plc1 ранее установленный режим «force» для переменных с именами do0 и cloop1. Если указанные переменные не находились в режиме «force», команда не имеет эффекта, но не считает эту ситуацию ошибочной. При отсутствии ошибок команда не выдает сообщений и завершается в кодом 0.

```
$ lbcmd plc1 -U do0=1 cloop1=0.004
```

3.3.5. Системные переменные

Имена вида sys.XXX зарезервированы для системных переменных, позволяющих получить информацию об указанном модуле ПЛК или изменить режим его работы. Сводный список системных переменных приведен в Таблице 1; конкретный набор переменных свой у каждого типа модуля.

Таблица 1 – Системные переменные

Имя переменной	Режим доступа	Описание
sys.alarm	RO	биты ошибок головного модуля
sys.alarmN	RO	биты ошибок модуля номер N
sys.devtype	RO	тип (артикул) модуля, например LB241BC
sys.macaddr	RO	MAC-адрес (уникальный идентификатор EUI-48) модуля
sys.run	RW	разрешение исполнения программ plcprog (1=разрешено, 0=запрещено)
sys.serial	RO	серийный номер модуля

см. далее

Продолжение таблицы 1

Имя переменной	Режим доступа	Описание
<code>sys.status</code>	RO	биты состояния головного модуля
<code>sys.statusN</code>	RO	биты состояния модуля номер N
<code>sys.trace</code>	RW	задание режима низкоуровневой трассировки (только для разработчика)
<code>sys.uptime</code>	RO	время от последней перезагрузки модуля, с
<code>sys.version</code>	RO	версия встроенного ПО модуля
<code>sys.wdt_pause</code>	WO	запрет рассылки уведомлений сторожевым таймерам подчиненных модулей на указанный период (мс), чтобы вызвать аппаратный сброс модуля, с которым потеряна связь

3.3.6. Команда `calibrate`

Команда `./lbcmd PLCNAME/SLOT --calibrate` запускает процедуру калибровки для модулей, предусматривающих возможность калибровки. Команда не предназначена для конечного пользователя; процедура калибровки проводится на заводе-изготовителе.

3.3.7. Команда `fboot`

Команда `lbcmd plc1 --fboot forte.fboot` загружает в постоянную память базового модуля ПЛК с именем `plc1` файл `forte.fboot` с проектом, подготовленным в 4diac IDE. ПЛК начнет выполнять проект в подсистеме 4diac FORTE после перезагрузки или выключения-включения питания. Перезагрузку отдельного модуля можно выполнить удаленно с помощью команды `restart` (раздел 3.3.11).

3.3.8. Команда `getconf`

Команда `lbcmd plc1 --getconf` считывает и выводит на стандартный вывод конфигурацию, загруженную на данный момент в ПЛК с именем `plc1`, в формате YAML.

3.3.9. Команда `json`

Команда `lbcmd plc1 --json` выводит на стандартный вывод в формате JSON конфигурацию ПЛК с именем `plc1`, загруженную из файла или каталога `conf.yml` (раздел 5). Команда не предназначена для конечного пользователя.

3.3.10. Команда `nofboot`

Команда `lbcmd plc1 --nofboot` удаляет из постоянной памяти ПЛК проект `4diac`, ранее загруженный командой `fboot`.

3.3.11. Команда `restart`

Команда `lbcmd plc1/3 --restart 10s` устанавливает на модуле в слоте 3 ПЛК с именем `plc1` таймер перезагрузки: модуль перезагрузится через 10 секунд. В аргументе команды допускаются обычные суффиксы времени: `ms`, `s`, `m`, `h`, `d`; при отсутствии суффикса предполагается `s` (секунды).

Повторный вызов команды `--restart` до срабатывания таймера обновляет счетчик таймера, т.е. откладывает перезагрузку. Периодический вызов команды `--restart` позволяет получить режим «сторожевого таймера»: ПЛК перезагрузится, если ни одна команда `restart` не поступила в течение заданного интервала.

Команда `lbcmd plc1 --restart 0` останавливает ранее запущенный таймер, а если таймер не был запущен, ничего не изменяет:

Значения задержки $\leq 0.1s$ обрабатываются особо: в случае ошибки таймерной службы ПЛК, перезагрузка выполняется немедленно, без использования таймера.

3.3.12. Команда `settime`

Команда `lbcmd plc1 --settime` устанавливает системное время ПЛК с именем `plc1` равным системному времени управляющего компьютера. Время устанавливается с точностью до задержки передачи команды с компьютера на ПЛК.

Предпочтительным способом установки времени ПЛК является настройка синхронизации времени по протоколу SNTP (раздел 5.2.1), поскольку в

этом случае время не только однократно устанавливается, но его точность поддерживается периодической синхронизацией.

3.3.13. Команда `stats`

Команда `lbcmd plc1/3 --stats` выводит счетчики событий, накопленные модулем в слоте 3 ПЛК `pl1` со времени последней перезагрузки ПЛК. Значения счетчиков могут быть полезны для диагностики. У каждого типа модуля свой набор счетчиков, по которым можно судить о наличии и частоте ошибок в работе модуля или об интенсивности использования определенных частей кода встроенного ПО модуля. Выводятся только счетчики, имеющие ненулевые значения.

3.3.14. Опциональные аргументы программы `lbcmd`

`--help` программа `lbcmd` выдает краткую справку по использованию и завершается:

```
$ lbcmd --help
Usage: lbcmd PLCNAME[/SLOT] [SECONDS] VAR [VAR...]
       lbcmd PLCNAME[/SLOT] [SECONDS] VAR=VAL [VAR=VAL...]
       lbcmd PLCNAME[/SLOT] [SECONDS] [-F|--force] VAR=VAL [VAR=VAL...]
       lbcmd PLCNAME[/SLOT] [SECONDS] [-U|--unforce] VAR [VAR...]
       lbcmd PLCNAME[/SLOT] --calibrate SPEC
       lbcmd PLCNAME[/SLOT] --fboot FILE
       lbcmd PLCNAME[/SLOT] --getconf
       lbcmd PLCNAME[/SLOT] --json
       lbcmd PLCNAME[/SLOT] --nofboot
       lbcmd PLCNAME[/SLOT] --restart {10s|1h|...}
       lbcmd PLCNAME[/SLOT] --settime[=SECONDS]
       lbcmd PLCNAME[/SLOT] --stats
```

`-f /path/to/conf.yml` задает альтернативный путь к файлу конфигурации. По умолчанию программа `lbcmd` использует файл конфигурации `BASEDIR/conf/conf.yml`, который находится исходя из пути запуска программы `lbcmd(BASEDIR/bin/lbconf)`;

`--bymas` устанавливает режим обращения к ПЛК по локальному (`link local`) адресу IPv6, даже если для целевого ПЛК в файле `conf.yml` задан статический адрес IPv4. См. раздел 3.2.2;

`SECONDS` задает интервал повторения команды и может быть любым действительным числом, например 10 или 0.5. При отсутствии аргумента

SECONDS команда выполняется однократно и завершается; при наличии аргумента SECONDS она выполняется до нажатия Ctrl-C пользователем.

3.4. Программа lbota

Программа lbota выполняет загрузку новой версии встроенного ПО в указанный модуль ПЛК Logic Box, используя протокол управления NPC21_Modbus.

3.4.1. Формат обращения

```
$ lbota PLCNAME[/SLOT[/ADDR]] [опции] /path/to/firmware.bin[.xz]
```

Аргумент PLCNAME/SLOT задает имя ПЛК и номер слота модуля, в который будет загружено ПО. Имя ПЛК должно присутствовать в конфигурационном файле conf.yml, описание слота может отсутствовать. Если опциональная часть [/SLOT] не указана, обращение происходит к головному (master) модулю указанного ПЛК. Опциональная часть /ADDR применяется для обращения к устройству, подключенному к внешней шине Modbus модуля LB241MG.

Файлы для загрузки публикуются изготовителем в составе ПО Logicbox [9], в каталоге logicbox/firmware.

3.4.2. Опциональные аргументы программы lbota

--help: программа lbota выдает краткую справку по использованию и завершается:

```
Usage: lbota PLCNAME[/SLOT[/ADDR]] [--rewind] /path/to/firmware.bin  
lbota ... [-f conffile.yml] ...
```

-f /path/to/conf.yml задает альтернативный путь к файлу конфигурации. По умолчанию программа lbota использует файл конфигурации *BASEDIR/conf/conf.yml*, который находится исходя из пути запуска программы *lbota(BASEDIR/bin/lbconf)*;

--rewind: опция позволяет начать загрузку файла сначала. Если опция --rewind не указана, загрузка продолжается с места, где она была остановлена

оператором (Ctrl-C) или из-за перебоев в связи, если размер загружаемого файла одинаковый в текущем и прошлом обращении к программе lbota.

3.4.3. Примеры обращения

Загрузка ПО в модуль аналогового ввода LB241AI4, находящийся в позиции 3 в сборке УСО/ПЛК LB241, присутствующего в конфигурационном файле `conf.yml` с именем bc3:

```
$ lbota bc3/3 ~/logicbox/firmware/LB241AI4
```

3.5. Программа lblog

Программа lblog позволяет считывать содержимое журнала системных сообщений указанного модуля ПЛК Logic Vox в целях расширенной диагностики. Журнал организован как FIFO-буфер текстовых строк; объем журнала зависит от типа модуля и варьируется от 20КБ до 2МБ. Поддерживается как считывание журнала целиком, так и выборка строк, удовлетворяющих заданному образцу.

3.5.1. Формат обращения

```
lblog PLCNAME[/SL0T[/ADDR]] [опции] [REGEX ...]
```

Аргумент PLCNAME/SL0T задает имя ПЛК и номер слота модуля, к журналу которого происходит обращение. Имя ПЛК должно присутствовать в конфигурационном файле `conf.yml`, описание слота может отсутствовать. Если опциональная часть [/SL0T] не указана, обращение происходит к головному (master) модулю указанного ПЛК. Опциональная часть /ADDR применяется для обращения к устройству, подключенному к внешней шине Modbus модуля LB241MG.

Опциональный аргумент REGEX позволяет уменьшить объем передаваемой информации, включая в выборку только записи журнала, удовлетворяющие заданному образцу – регулярному выражению REGEX. Описание синтаксиса регулярных выражений POSIX можно найти в документации ОС GNU/Linux: `man 7 regex` или [10].

Если задано несколько аргументов REGEX, результаты сопоставления объединяются логическим И.

3.5.2. Опциональные аргументы программы lblog

--help: программа lblog выдает краткую справку по использованию и завершается:

```
Usage: lblog PLCNAME [[-eirsv] REGEX ...] [-N] [-f]
  REGEX is a POSIX regular expression (man 7 regex)
  N is the number of most recent records to fetch
  -e use POSIX Extended Regular Expression syntax
  -f `follow' mode, like tail -f
  -i ignore case
  -r output in reverse chronological order
  -s let '.' match newline (\n)
  -v invert match result, like grep -v
```

-f /path/to/conf.yml задает альтернативный путь к файлу конфигурации. По умолчанию программа lblog использует файл конфигурации *BASEDIR/conf/conf.yml*, который находится исходя из пути запуска программы *lblog(BASEDIR/bin/lbconf)*;

-e: включает расширенный синтаксис регулярных выражений [10];

-f: включает «режим сопровождения»: после вывода всех заданных другими аргументами записей программа lblog не завершается, а продолжает работать, выводя новые записи по мере их добавления в журнал;

-i: делает образец REGEX нечувствительным к регистру букв (case-insensitive);

-r: выводит записи в обратном хронологическом порядке, начиная с самой недавней;

-s: разрешает метасимволу '.' в регулярных выражениях сопоставляться с символом перевода строки (LF, '\n') в многострочных записях журнала;

-v: инверсия результата сопоставления записи журнала с образцом: строка, удовлетворяющая регулярному выражению, будет не включена, а наоборот пропущена при выводе.

3.5.3. Примеры обращения

Считывание и сохранение в файле `bc3.log` всех записей журнала модуля аналогового ввода LB241AI4, находящегося в позиции 3 в сборке УСО/ПЛК LB241, присутствующего в конфигурационном файле `conf.yml` с именем `bc3`:

```
$ lblog bc3/3 > bc3.log
```

Считывание трех последних записей журнала головного модуля ПЛК `plc103`, после чего ожидание новых записей и вывод их по мере поступления:

```
$ lblog plc103 -3f
```

Считывание всех записей журнала модуля аналогового ввода LB241AI4, кроме относящихся к каналам аналогового ввода номер 0 и номер 1 или записанных функцией `app_main`:

```
$ lblog bc3/3 -v 'chan[01]' -v app_main
```

3.6. Программа lbupload

Программа lbupload выполняет компиляцию и загрузку в ПЛК программ на языке ST IEC 61131-3. Загрузка программы не требует перезагрузки ПЛК и может выполняться на фоне с выполнения в ПЛК ранее загруженных программ, подобно запуску команд в командной строке операционной системы.

Для компиляции программа ПЛК должна быть описана в файле `conf.yml` в разделе `plcN:plcprogN`, где в частности указывается имя файла, содержащего текст программы на языке ST. Для запуска программы на исполнение она должна быть загружена программой lbupload, и в конфигурации должно быть разрешено ее исполнение (`plcN:plcprogN:enable:y`). Язык конфигурации подробно описан в разделе 5.

Программа lbupload требует два обязательных аргумента: имя ПЛК и индекса программы `plcprogN`. Допускается краткая запись индекса в виде `N`.

```
$ lbupload plc1 plcprog1
segment sizes: textsz=992 datasz=64 bsssz=160
GETPLACE returned text=0x42890000 data=0x3c7fffa0 bss=0x3c7ffef8
/home/user/elprivod/st/test1.txt upload as prog1 successful.
```

При таком запуске программы lbupload происходит компиляция программы `plcprog1`, при отсутствии ошибок компиляции загрузка программы в ПЛК. После успешной загрузки происходит запуск программы на исполнение, если в конфигурации присутствует параметр `plcN: plcprogN: enable: y`,

3.6.1. Опциональные аргументы программы lbupload

`--help`: команда lbupload выдает краткую справку по использованию и завершается:

```
$ lbupload --help
Usage: lbupload PLCNAME plcprogN
       lbupload -c PLCNAME plcprogN
       lbupload -c FILENAME.st
       lbupload [-f conffile.yml] ...
```

`-f /path/to/conf.yml` задает альтернативный путь к файлу конфигурации. По умолчанию программа lbupload использует файл

конфигурации *BASEDIR/conf/conf.yml*, который находится исходя из пути запуска программы *lbupload(BASEDIR/bin/lbconf)*;

-с: опция запрещает загрузку программы в ПЛК; выполняется только компиляция программы с выводом размера секций программы и сообщений об ошибках, если в тексте программы на языке ST есть ошибки. При вызове *lbupload* с ключом -с допускается передача в качестве аргумента имени файла с программой вместо имени ПЛК и индекса программы, например: *upload -c test.st*.

3.6.2. Замечания по реализации

Программа *lbupload* в ходе работы несколько раз вызывает системную утилиту *make* в каталоге *BASEDIR/st*, задавая ей разные цели компиляции и параметры. Процесс компиляции определен в файле *BASEDIR/st/Makefile*; этот файл не предполагает изменения пользователем, но может быть полезен для ознакомления с деталями процесса компиляции.

Компиляция и загрузка программы происходит в пять этапов:

1) Трансляция с языка ST в язык Си транслятором *maties* [3] (исполняемый файл транслятора *ies2c*). Результат трансляции представлен в виде ряда файлов в каталоге *BASEDIR/st/build*:

-rw-r--r--	1	npc21live	npc21live	617	Sep 24 18:41	Config0.c
-rw-r--r--	1	npc21live	npc21live	1	Sep 24 18:41	Config0.h
-rw-r--r--	1	npc21live	npc21live	108	Sep 24 18:41	LOCATED_VARIABLES.h
-rw-r--r--	1	npc21live	npc21live	22	Sep 24 18:41	POUS.c
-rw-r--r--	1	npc21live	npc21live	122	Sep 24 18:41	POUS.h
-rw-r--r--	1	npc21live	npc21live	1344	Sep 24 18:41	program0.c
-rw-r--r--	1	npc21live	npc21live	477	Sep 24 18:41	program0.h
-rw-r--r--	1	npc21live	npc21live	677	Sep 24 18:41	Res0.c
-rw-r--r--	1	npc21live	npc21live	2253	Sep 24 18:41	VARIABLES.csv

2) Предварительная компиляция из языка Си в машинный код с использованием программ из комплекта GNU Toolchain, входящих в поставку SDK *esp-idf*. Главная из этих программ – кросс-компилятор языка Си для архитектуры Xtensa, *xtensa-esp32s3-elf-gcc*. В результате компиляции создаются файлы:

build/text.bin с образом сегмента команд;

build/data.bin с образом сегмента инициализированных данных;

build/map.txt с картой памяти скомпилированной программы.

Файлы text.bin и data.bin являются абсолютными загружаемыми образами: они не содержат информации о перемещении, а настроены на загрузку по заранее определенным адресам сегментов text, data и bss. «Предварительность» данного этапа компиляции состоит в том, что конкретные адреса загрузки пока не известны, они задаются произвольно. Цель предварительной компиляции – определить размеры сегментов text, data и bss.

Если программа lbrupload запущена с ключом -c («только компиляция»), на этом этапе она завершается.

3) Определение адресов загрузки. Программа lbrupload обращается к ПЛК по протоколу Modbus TCP с использованием нестандартной (User defined) функции NPC21_MODBUS, в которой передается команда GETPLACE с размерами сегментов .text, .data и .bss в качестве аргументов. ПЛК выделяет память для загрузки программы с указанными размерами. Результатом команды являются адреса выделенных сегментов памяти.

4) Окончательная компиляция: повторяется процесс компиляции из языка Си в машинный код процессора Xtensa с генерацией абсолютных образов сегментов, настроенных на полученные в ответе на команду GETPLACE адреса загрузки сегментов .text, .data и .bss.

5) Загрузка программы. Программа lbrupload обращается к ПЛК по протоколу NPC21_MODBUS, в которой передается команда PLCPRG. Аргументом команды являются заголовок с размерами сегментов, бинарные образы сегментов .text и .data и контрольные суммы сегментов (SHA256), защищающие загружаемые образы от ошибок передачи. При отсутствии ошибок передачи бинарные образы загружаются в области памяти, выделенные по команде GETPLACE.

После успешной загрузки программа немедленно запускается на исполнение, если выполнены остальные необходимые условия запуска (раздел 2.1.2).

4. ВХОДНЫЕ И ВЫХОДНЫЕ ДАННЫЕ

Для всех утилит ПО управления (`lbconf ... lbupload`) входными данными является конфигурационный файл `conf.yml`. Формат конфигурационного файла описан в разделе 5.

Для программы `lbupload` входными данными является текст загружаемой программы на языке ST. Язык ST детально описан в стандарте IEC 61131-3 [6] и в настоящем документе не рассматривается.

Выходными данными для всех программ, входящих в состав «ПО управления», являются сообщения, выдаваемые программами на стандартный вывод и стандартный вывод ошибок ОС Linux. Сообщения, выдаваемые программами при нормальной работе, описаны в разделе 3. Сообщения, выдаваемые программами при возникновении ошибок, описаны в разделе 7.

5. КОНФИГУРАЦИЯ ПЛК LOGIC BOX

5.1. Конфигурационный файл `conf.yml`

Конфигурационный файл `conf.yml` является входными данными для всех утилит ПО управления (`lbconf ... lbupload`). Файл `conf.yml` содержит древовидную структуру конфигурационных данных в формате YAML [2] с ассоциативным массивом (ключ–значение) на верхнем уровне. Ключи ассоциативного массива на первом (верхнем) уровне дерева это имена устройств (ПЛК или УСО) в распределенной системе автоматизации. Ключи на втором уровне дерева это имена разделов конфигурации. Пример:

```
bc1:
  clock: {...}
  slot-1: {...}
  slot1: {...}
  slot2: {...}
plc202:
  clock: {...}
  modbus\_client: {...}
  task0: {...}
  task1: {...}
  slot-2: {...}
  slot-1: {...}
  slot1: {...}
  slot2: {...}
...
```

Раздел с именем вида `slotN` описывает конфигурацию модуля в слоте (разъеме расширения) номер `N` указанного ПЛК. Остальные разделы на уровне 2 описывают конфигурацию программных подсистем, относящуюся не к конкретному слоту, а к ПЛК в целом; они интерпретируются базовым модулем, который находится в роли головного.

5.1.1. Многофайловая конфигурация

При большом объеме конфигурационных данных можно представить их не в одном файле `conf.yml`, а в виде файлового дерева. Пример из раздела 5.1 можно представить следующим файловым деревом:

```
conf.yml/bc1.yml
conf.yml/bc1/slot-1.yml
conf.yml/bc1/slot1.yml
```

```
conf.yml/bc1/slot2.yml  
conf.yml/plc202.yml  
conf.yml/plc202/slot-2.yml  
conf.yml/plc202/slot-1.yml  
conf.yml/plc202/slot1.yml  
conf.yml/plc202/slot2.yml
```

Можно выбрать и другие варианты разбиения дерева конфигурации на файлы, зная правила чтения дерева конфигурации:

- 1) разбор начинается с имени `conf.yml`, будь то файл или каталог;
- 2) после чтения файла `X.yml`, дополнительно прочитывается каталог с именем `X`, если он существует;
- 3) при чтении каталога прочитываются все записи из него с суффиксом `.yml` (как файлы так и каталоги). Прочитанное из файла/каталога `Y.yml` поддереву конфигурационных данных добавляется в вышестоящее дерево как элемент в ассоциативном массиве с ключом `Y`;

5.2. Разделы конфигурации

5.2.1. Раздел `clock`

Конфигурация клиента протокола синхронизации времени SNTP. Клиент SNTP работает на головном модуле ПЛК, остальные модули получают сигналы точного времени от головного модуля по шине ПЛК.

```
clock:  
  sntp_server: pool.ntp.org  
  sntp_server2: ntp.ix.ru  
  sntp_smooth: y # y=use adjtime, n=use settimeofday
```

5.2.2. Раздел `forte`

Раздел конфигурации подсистемы 4diac FORTE. Устанавливает разрешение выполнения подсистемы `enable: y` и привязывает переменные брокера к функциональным блокам PUBLISH и SUBSCRIBE в проекте 4diac.

Пример:

```
forte:  
  enable: y  
  var: [from_forte1, from_forte2..3, cmd, do1]
```

```
var_out: [to_forte1, to_forte2..3, cmd]
```

5.2.3. Раздел modbus_client

Конфигурация экземпляра клиента протокола Modbus. Возможно присутствие более одного раздела modbus_clientN, явного ограничения на количество разделов modbus_clientN нет.

Функция Modbus выбирается автоматически в зависимости от типа регистра (coil, inbit, holding, inreg).

Пример: клиент Modbus TCP (задан параметр server)

```
modbus_client: # эквивалентно modbus_client0
  server: 192.168.1.60:1502 # адрес и порт сервера
  # запись 40 регистров fn=0x10 Write Multiple Registers
  holding0..39: {var_out: VALVE1..40, period: 1s}
modbus_client2: # N=2, должно отличаться от других modbus_clientN
  server: 192.168.1.610 # адрес сервера. Порт 502 по умолчанию
  # чтение одного регистра 300h, fn=0x04 Read Input Registers
  inreg0x300: {var: sensor5, period: 1.5s}
```

Пример: клиент Modbus RTU (задан параметр addr). Конфигурация клиента размещена в разделе slotN модуля, к порту RS-485 которого подключен нужный сегмент Modbus.

```
slot-1:
  modbus_client100:
    addr: 2 # адрес сервера RTU, 1..247
    holding4..23: {var: mctest204..223, period: 10ms}
    holding204..205: {var_out: mctest204..205, period: 2s}
```

5.2.4. Раздел modbus_server

Директива задает конфигурация сервера Modbus TCP IPv4 и Modbus RTU. Сервер Modbus TCP IPv6 не требует конфигурации, запускается всегда на порту 502 и используется для конфигурации.

Пример: сервер Modbus TCP (задан параметр tcp_port)

```
modbus_server:
  tcp_port: 502 # optional; if present, enables Modbus TCP
  # регистр 0001 отвечает на fn=0x3 Read Holding Registers,
  # fn=0x06 Write Single Register, fn=0x10 Write Multiple registers
  holding1: {var_out: cmd, var: cmd}
```

```
# регистры 0004, 0005, 0006, 0007 отвечают на fn=0x3 Read Holding Registers
holding4..7: {var_out: ai0..3}
fussy_read: n # неопределенный регистр читается как 0
fussy_write: y # запись неопределенного регистра дает ошибку Modbus
```

5.2.5. Раздел slot

Раздел slotN описывает конфигурацию модуля в слоте расширения номер N ПЛК LB340, или в позиции N в сборке YCO LB241.

Пример: конфигурация модуля дискретных выходов

```
slot1:
  module: LB241D016
  chan0..15:
    var_out: do0..15
```

Пример: конфигурация дополнительного базового модуля. Включает конфигурацию клиента Modbus RTU, подключенного к порту RS-485 данного модуля.

[illegible]

5.2.6. Раздел var

В разделе `var` можно задавать начальные значения и атрибуты глобальных переменных ПЛК, для которых это требуется в конкретном приложении. Не все переменные необходимо перечислять в разделе `var`, но если переменная упоминается как `VAR_EXTERNAL` в программах ПЛК, она должна присутствовать и иметь атрибут `init`.

Пример:

```
var:
  # 01-oscillator-stop
```

```
start: {retain: y, init: 0}  
out1: 1 # упрощенный синтаксис; эквивалентно {init: 1}  
stop0: 0  
# snake  
exec: 1  
mw1: 0  
inc1: 0  
stop1: 0  
cmd: {multisource: y, init: 0}
```

Атрибут `retain: y` устанавливает режим сохранения данной переменной в энергонезависимой памяти. Последнее записанное значение будет восстановлено после перезагрузки.

Атрибут `multisource: y` отменяет для данной переменной общее правило «у переменной только один издатель» (раздел 2.1.1). При единственном издателе системное ПО гарантирует, что после обновления переменной все подсистемы ПЛК получат ее новое значение (почти одновременно, с небольшой вариацией задержки). После включения режима `multisource` этой гарантии больше нет: если два издателя публикуют новые значения в быстрой последовательности, возможны нежелательные эффекты, вплоть до разных значений переменной в разных подсистемах. Корректность работы в режиме `multisource` гарантируется, если промежуток времени между обновлениями переменной больше, чем время распространения по всем подсистемам ПЛК (которое зависит от аппаратной конфигурации конкретного ПЛК). Ответственность за соблюдение этого условия возлагается на прикладного программиста.

5.2.7. Раздел `wdt`

Настройка параметров сторожевых таймеров («watchdog timers»). В каждом модуле ПЛК Logic Box работает сторожевой таймер, способный выполнить аппаратный сброс модуля для автоматического восстановления работоспособности модуля после сбоя. При нормальной работе головной модуль ПЛК регулярно рассылает сообщения с указанием периода t , при получении которых сторожевой таймер откладывает сброс на t секунд. Если какой-то модуль из-за сбоя не получает этих сообщений или не может их обработать

(«подвис», перегружен и т.д.), сторожевой таймер сбрасывает модуль, модуль перезагружается и возвращается в работу.

Пример:

<pre>wdt: bus: 10s # период сторожевого таймера в подчиненных модулях ПЛК self: 30s # период сторожевого таймера в головном модуле ПЛК</pre>

При отсутствии в конфигурации раздела `wdt` сторожевой таймер в базовых модулях настраивается на период $t = 3600\text{с}$, в модулях ввода/вывода на период $t = 300\text{с}$.

5.3. Параметры конфигурации

В настоящем разделе в алфавитном порядке описываются параметры (атрибуты) конфигурации, которые могут встречаться в разных разделах на разных уровнях дерева конфигурации. Как правило, параметр с одним именем имеет один и тот же смысл независимо от раздела, где он встречается, поэтому его достаточно описать один раз. В редких случаях, когда параметр в разных разделах интерпретируется по-разному, он описывается несколько раз с указанием контекста.

Значение параметра, применяемое по умолчанию, приводится в утолщенной рамке.

5.3.1. Параметр `direct (slot:iec104_server:io)`

direct: y объект `io` протокола IEC-104 воспринимает прямые (непосредственные) команды.

`direct: n` объект `io` протокола IEC-104 воспринимает только команды с предварительным выбором ([11], 6.8.1). На прямые (непосредственные) команды отвечает отрицательным подтверждением `ACTCON`.

5.3.2. Параметр `interval`

Задаёт минимальный временной интервал между действиями для объекта, в котором находится параметр.

Пример: ограничение максимальной частоты отправки значений модулем аналогового ввода при быстрых изменениях напряжения на входе:

```
module: LB241AI4
chan1:
  mode: 0-10V
  resolution: 0.0001      # игнорировать изменения менее 0.1мВ
  interval: 0.1s          # максимальная частота отправки 10Гц
  period: 0.5s            # минимальная частота отправки 2Гц
  var: sensor1
```

5.3.3. Параметр module (slot)

Обязательный параметр каждого раздела slotN, указывающий тип (артикул) модуля, установленного в слот N (или находящегося в позиции N в сборке УСО LB241). Каждым модуль при загрузке конфигурации проверяет значение параметра на соответствие своему типу; в случае несовпадения конфигурация не принимается.

Пример: `module: LB241AI4`

5.3.4. Параметр macaddr (slot)

Обязательный параметр для разделов slot, в которые установлены модули с интерфейсом Ethernet; используется для формирования адреса IPv6 link local при обращениях к модулю с управляющего компьютера.

Пример: `macaddr: f4:12:fa:d4:d5:7f`

5.3.5. Параметр quality (slot:iec104_server:command)

`quality: n`

`quality: y`

Включает для конкретного объекта ИЕС-104 режим добавления описателя качества ([12], 7.2.6.3) при передаче данного объекта по протоколу МЭК-104. Биты описания качества происходят от источника значения переменной: например, программы ПЛК или объекта ioN, являющегося источником данной переменной. Если источник значения переменной (например, modbus_server)

не поставляет битов качества, передается нулевой описатель, означающий наилучшее качество значения.

5.3.6. Параметр `quality (slot:iec104_server:meas)`

См. 5.3.5

5.3.7. Параметр `quality (slot:iec104_server)`

Задаёт значение параметра `quality` по умолчанию для всех объектов `io` данного сервера протокола IEC-104. См. 5.3.5.

5.3.8. Параметр `select (slot:iec104_server:io)`

Параметр `select` управляет обработкой входящих команд в режиме «выбора и исполнения» ([11], 6.8.1). Входящее сообщение с установленным битом `S/E` ([12], 7.2.6.26) воспринимается как директива предварительного выбора (`SELECT`).

`select: auto` модуль немедленно отвечает на директиву выбора положительным подтверждением (`ACTCON`) и начинает отсчет времени (`select_timeout`), в течение которого должна прийти выбранная команда (`EXCO`). Если до поступления команды поступит новая директива выбора для того же объекта `io`, модуль ответит отрицательным подтверждением `ACTCON`. После поступления команды передается подтверждение (положительное, если данные в команде совпадают с данными предварительного выбора), объект `io` выходит из состояния ожидания команды и при поступлении новой директивы выбора снова ответит положительным подтверждением `ACTCON`.

`select: ind` модуль сигнализирует о возникшем событии выбора программам ПЛК, работающим на базовом модуле, путем передачи предлагаемого нового значения объекта `io` в атрибуте `select_ind` переменной объекта (`io:var:name`). Программа ПЛК подтверждает готовность к исполнению команды (`EXCO`) установкой атрибута `select_res` равным 1, или сообщает о неготовности установкой атрибута `select_res` равным 0. Механизм подтверждения работает корректно, даже если новое значение `select_ind`

совпадает с значением, установленным предыдущей командой, поскольку программа работает по событиям изменения атрибутов, которые генерируются даже если новое значение атрибута совпадает с предыдущим.

В случае получения отрицательного подтверждения `select_res=0` модуль отвечает на директиву выбора отрицательным подтверждением `ACTCON` и объект `io` становится готов к приему новой директивы выбора.

В случае получения подтверждения `select_res=1` модуль отвечает на директиву выбора положительным подтверждением (`ACTCON`) и начинает отсчет времени `select_timeout` (5.3.9), в течение которого должна прийти выбранная команда (`EXCO`). Если до поступления команды поступит новая директива выбора для того же объекта `io`, модуль ответит отрицательным подтверждением `ACTCON`. После поступления команды объект `io` выходит из состояния ожидания команды и становится готов к приему новой директивы выбора.

См. также: `direct` (5.3.1), `select_timeout` (5.3.9).

5.3.9. Параметр `select_timeout (slot:iec104_server:io)`

<code>select_timeout: 5s</code>

Задает время ожидания команды (`EXCO`), см. 5.3.8.

5.3.10. Параметр `select_timeout (slot:iec104_server)`

При использовании в контексте `slot:iec104_server` параметр влияет на все объекты `io` данного сервера, не имеющие собственной установки `select_timeout`. См. 5.3.9.

5.3.11. Параметр `period (task)`

Задает временной интервал исполнения программы ПЛК. По смыслу соответствует параметру `CONFIGURATION : RESOURCE : TASK : INTERVAL` стандарта IEC 61131 (заменяет его). Пример:

<code>task0: period: 10ms</code>

5.3.12. Параметр period (в других контекстах)

Задаёт временной интервал между регулярными действиями для объекта, в котором находится параметр. Специальное значение **period: 0** выключает регулярные действия для объекта.

Пример: период опроса сервера клиентом Modbus (5.2.3):

```
modbus_client5:
  period: 1s
```

Пример: период отправки значения модулем аналогового ввода при постоянном напряжении на входе:

```
module: LB241AI4
chan1:
  mode: 0-10V
  resolution: 0.0001      # игнорировать изменения менее 0.1мВ
  interval: 0.1s          # максимальная частота отправки 10Гц
  period: 0.5s            # минимальная частота отправки 2Гц
  var: sensor1
```

5.4. Примеры конфигурации

Листинг 1 – Фрагмент: конфигурация модуля IEC-104

```
1 slot7:
2   module: LB340IEC104
3
4   # MAC-адрес данного модуля, обязательный параметр
5   macaddr: f4:12:fa:d4:d5:7f
6
7   # опциональная конфигурация статических параметров TCP/IP
8   # если задана, отключает DHCP
9   #ipaddr: 192.168.1.90/24
10  #gateway: 192.168.1.60
11
12  # конфигурация первого сервера МЭК-104.
13  # Суффикс (в данном примере "1") используется как <<общий адрес ASDU>>
14  # ("common address", CA) данного сервера
15  iec104_server1:
16    # TCP-порт сервера, по умолчанию 2404
17    #tcp_port: 2404
18
19    # ограничение на число одновременно подключенных клиентов
20    # по умолчанию 50
21    #max_connections: 50
22
23    # опциональный список дозволенных клиентов
24    # по умолчанию принимаются все соединения
25    #allow: [192.168.1.60, 192.168.1.61]
```

```

26
27 # параметры уровня соединения МЭК-104 (APCIP)
28 conn_parms:
29     k: 12 # сколько максимум пакетов разрешено послать, не получая ask
30     w: 8 # приемник посылает ask, получив w пакетов (w <= 2k/3).
31     t0: 10s # тайм-аут на установление соединения
32     t1: 15s # тайм-аут ожидания подтверждения (ask)
33     t2: 10s # после t2 приемник посылает ask, даже если N < w
34     t3: 20s # период отправки ``поддерживающих`` сообщений
35             # при отсутствии другой активности
36
37 # параметры прикладного уровня
38 #app_parms:
39 # type_id_size: 1
40 # vsq_size: 1
41 # cot_size: 2
42 # originator_address: 0
43 # ca_size: 1
44 # ioa_size: 3
45 # asdu_maxsize: 249
46
47 # Прием C_CS_NA_1(103) | Clock synchronization command
48 clock_sync:
49     #period: 0 # 0 = не посылать никогда
50     timeout: 1h # считать время недействительным через 1 час
51                # после последней синхронизации.
52
53 #
54 # описание информационных объектов задает отображение пространства
55 # имен переменных брокера ПЛК на пространство адресов объектов (IOA)
56 #
57 # ioN: суффикс N представляет собой IOA.
58 # Допустимые значения N: 0..16777215 (2^24-1).
59 #
60 # ioN описывают обработку принимаемых сообщений типа `измерение'
61 # (measurement) и `управление' (command). Имеют атрибут `var:'.
62 #
63 # measN описывают отправляемые сообщения типа measurement.
64 # Имеют атрибут `var_out:'.
65 #
66 # commandN описывают отправляемые сообщения типа command.
67 # Имеют атрибут `var_out:'.
68 #
69
70 # M_ME_TF_1(36) Short measured value (FLOAT32) with CP56Time2a
71 # Периодическая передача не включена (нет атрибута `period:')
72 # Спорадическая передача не включена (нет атрибута `interval:')
73 # Сообщение типа measurement с IOA=5000 отправляется клиенту
74 # только по запросам Read или Interrogation
75 meas5000:
76     var_out: myvar1
77
78 # все возможные типы: singlepoint doublepoint stepposition
79 #                               bitstring normalized scaled float integrated
80 type: float
81
82 # возможные форматы времени: auto, 56, none

```

```

83      # 56 означает время в формате CP56Time2A,
84      # auto означает none при ответе на запрос Read, и "56" при
85      # периодической или спорадической передаче.
86      # по умолчанию auto.
87      time: 56
88
89      # M_ME_TF_1(36) Short measured value (FLOAT32) with CP56Time2a
90      # присутствует атрибут `interval:', поэтому значение переменной
91      # myvar20 передается в сообщении типа measurement `спорадически' с
92      # IOA=70 при изменениях переменной, если изменение больше +-0.05
93      # по сравнению с последним отправленным значением, и не чаще чем
94      # раз в секунду.
95      meas70:
96          var_out: myvar20
97          type: float
98          range: 0.05
99          interval: 1s
100
101      # M_BO_TB_1(33) | Bitstring32 (32 bit bitstring) with CP56Time2a
102      # присутствует атрибут `interval:', поэтому значение переменной
103      # myvar30 передается `спорадически' с IOA=71 при изменениях
104      # переменной, если изменение больше +-0.05 по сравнению с
105      # последним отправленным значением, и не чаще чем раз в секунду.
106      # Сообщение также посылается периодически с интервалом 3 минуты,
107      # если значение переменной myvar30 долго не изменялось.
108      # Параметр `range:' обычно не нужен при работе с дискретными
109      # переменными но, если его указать, принимается во внимание.
110      meas71: {var_out: myvar30, type: bitstring, interval: 1s, period: 3m}
111
112      # C_SC_TA_1(58) | Single command (BOOLEAN) with CP56Time2a
113      # при получении такой команды от клиента, ее значение записывается
114      # в переменную `myvar40' и пересылается базовому модулю ПЛК,
115      # но не чаще чем раз в 100 мс.
116      io160000: {var: myvar40, type: singlepoint, interval: 0.1s}
117
118      # C_BO_TA_1(64) | Bitstring command (32 bit bitstring) with CP56Time2a
119      # при получении такой команды от клиента, ее значение записывается
120      # в переменную `myvar41', но
121      # при отсутствии параметра `interval:' или задании `interval: 0'
122      # спорадическая передача значения переменной по шине ПЛК или
123      # в другие объекты согласно var_out не разрешена.
124      # Значение может быть считано клиентами МЭК-104
125      # при помощи Read или Interrogation.
126      io16: {var: myvar41, type: bitstring}
127
128      # C_IC_NA_1(100) | Interrogation command
129      # Подраздел описывает как обработку поступившей от клиента команды
130      # общего опроса (General Interrogation, qoi=20), так и отправку
131      # команды общего опроса. В данном примере команда никогда не
132      # посылается, поскольку `period: 0'. В ответ на General Interrogation
133      # всегда отправляются пакеты measurement с типами из
134      # в подразделов meas, в данном примере meas5000 и meas70
135      interrogation:
136          period: 0 # never send interrogation of this type
137          list: [meas5000, meas70]
138
139      # C_IC_NA_1(100) | Interrogation command

```

```

140 # другие группы опроса: 1..16 (qoi=21..36)
141 # Данный подраздел описывает реакцию на опрос в группе 3
142 interrogation3:
143     list: [meas70, meas71]
144
145 # Длительности короткого и длинного импульсов для
146 # Qualifier Of Command (QOC)
147 # default pulse configuration, commandN can override
148 # TODO later
149 #pulse: {short: 5s, long: 10s}
150
151 # event queue mode, chosen internally in lib60870 with
152 # CONFIG_CS104_SUPPORT_SERVER_MODE_*
153 queue:
154     size: 1000 # default: 100
155     mode: single # messages distributed between clients
156     #mode: per_connection # messages copied to every client
157     #mode: # messages copied to redgroups and distributed within
158     # redgroup: [192.168.1.60, 192.168.1.61]
159     # redgroup2: [192.168.1.62, 192.168.1.63]
160
161 # можно сконфигурировать несколько серверов с разными CA
162 #iec104_server2:
163
164 iec104_client0:
165     # IP-адрес сервера (slave), к которому обращается данный клиент (master)
166     # порт по умолчанию 2404
167     server: 192.168.1.205:2404
168
169     # <<адрес источника>> (OA), 0..255
170     originator_address: 3
171
172 # connection level parameters (APCIP)
173 conn_parms:
174     k: 12 # сколько максимум пакетов разрешено послать, не получая ask
175     w: 8 # приемник посылает ask, получив w пакетов (w <= 2k/3).
176     t0: 10s # тайм-аут на установление соединения
177     t1: 15s # тайм-аут ожидания подтверждения (ask)
178     t2: 10s # после t2 приемник посылает ask, даже если N < w
179     t3: 20s # период отправки ``поддерживающих`` сообщений
180         # при отсутствии другой активности
181
182 # <<общий адрес>> (CA) сервера. При необходимости обращения к
183 # серверу с несколькими CA на одном IP-адресе следует создать
184 # несколько подразделов clientN с разными значениями CA.
185 common_address: 1
186
187 # Передача C_CS_NA_1(103) | Clock synchronization command
188 # Получение C_CS_NA_1(103) клиентом не предполагается,
189 # поэтому параметр `timeout:` не нужен
190 clock_sync:
191     period: 10m # sendClockSyncCommand every 10 minutes
192     # timeout: 1h
193
194 # C_RD_NA_1: Read command
195 # команды периодического запроса значений отдельных объектов
196 # общий адрес (CA)=1, адрес информационного объекта (IOA)=81

```

```
197 # Сервер ответит сообщением Measurement или Control
198 read81: { var: xyz1, period: 30s }
199 read1281: { var: abcd, period: 60s}
200
201 # C_IC_NA_1(100) | Interrogation command
202 # Передача команды общего опроса (general interrogation, qoi=20)
203 # каждые 10 минут
204 # Можно добавить атрибут `list:` чтобы задать реакцию на входящие
205 # команды опроса, но обычно клиенты не получают входящих
206 # команд опроса
207 interrogation: {period: 10m}
208
209 # C_IC_NA_1(100) | Interrogation command
210 # Передача команды опроса в группу3 (interrogation qoi=23)
211 # раз в минуту. Допустимые значения номера группы 1..16
212 interrogation3: {period: 1m}
213
214 # C_TS_TA_1 | Test command
215 # передача команды test раз в 30 секунд
216 test: {period: 30s, tsc: 0x4938}
217
218 # по умолчанию при отсутствии параметра `interval:`
219 # в ioN, measN, commandN спорадическая передача запрещена.
220 # Здесь можно указать значение интервала по умолчанию, чтобы
221 # разрешить спорадическую передачу сразу всех переменных,
222 # не задавая атрибут `interval:` для каждой переменной.
223 #interval: 1s
224
225 #
226 # Передача данных на сервер, спорадическая и периодическая
227 #
228
229 # C_SE_TC_1(63) | Setpoint command, short value (FLOAT32) with CP56Time2a
230 # Команда уставки с меткой времени CP56Время2а, короткий формат
231 # с плавающей запятой.
232 # Передается раз в 5 секунд
233 command5000:
234     var_out: myvar55
235     type: float
236     period: 5s
237
238 # C_BO_TA_1(64) | Bitstring command (32 bit bitstring) with CP56Time2a
239 # Строка из 32 битов с меткой времени CP56Время2а.
240 # Передается раз в 50 секунд
241 command5001:
242     var_out: myvar56
243     type: bitstring
244     period: 50s
245
246 # C_RC_TA_1(60) | Regulating step command
247 # команда пошагового регулирования с меткой времени CP56Время2а
248 # Передается спорадически, максимальная частота передачи 2Гц
249 command5010:
250     var_out: myvar60
251     type: stepposition
252     interval: 0.5s
253
```

```
254 # C_SC_TA_1(58) | Single command (BOOLEAN) with CP56Time2a
255 # одноэлементная команда с меткой времени CP56Время2a
256 # Передается спорадически, максимальная частота передачи 10Гц
257 command5011:
258     var_out: myvar61
259     type: singlepoint
260     interval: 0.1s
261
262 #
263 # Прием данных от сервера
264 # Передача принятых данных базовому модулю по шине ПЛК регламентируется
265 # параметрами `period:', `interval:' и `range:', как везде
266 #
267
268 # одна запись ioN задает обработку нескольких типов пакетов:
269 #
270 # M_ME_NC_1(13) | значение измеряемой величины, короткий формат
271 #                 с плавающей запятой
272 # M_ME_TF_1(36) | значение измеряемой величины, короткий формат
273 #                 с плавающей запятой с меткой времени CP56Время2a
274 # C_SE_NC_1(50) | команда уставки, короткое число с плавающей запятой
275 # C_SE_TC_1(63) | команда уставки, '-' с меткой времени CP56Время2a
276 io81:
277     var: myvar50
278     type: float
279     range: 0.1
280     interval: 10ms
281
282 # диапазоны переменных допустимы для io, meas, command, read.
283 #
284 # информационные объекты с адресами (IOA) от 200 до 210
285 # обрабатываются единообразно: присваиваются соответствующим
286 # переменным в диапазоне myvar55 ... myvar65.
287 # Длины диапазона объектов и переменных должны совпадать.
288 io200..210:
289     var: myvar55..65
290     # все возможные типы: singlepoint doublepoint stepposition
291     #                       bitstring normalized scaled float integrated
292     type: bitstring
293     interval: 1s
294
295 # альтернативный способ задания диапазона: НАЧАЛО+ДЛИНА
296 io220+5:
297     var: myvar120+5
298     type: singlepoint
299     interval: 1s
300
301 # допускается смешивать способ задания диапазонов, при условии
302 # что длины совпадают. Допускаются шестнадцатеричные числа.
303 io0x100+4:
304     var: myvar115..119
305     type: doublepoint
306     interval: 1s
307
308 # количество клиентов в конфигурации не ограничено явно, только
309 # объемом памяти.
310 #iec104_client2:
```

311	# ...
-----	-------

6. ДИАГНОСТИКА

Встроенное ПО каждого модуля ПЛК Logic Box формирует и регулярно передает на головной модуль слова состояния, содержащие биты состояния и ошибок. На головном модуле полученные от подчиненных модулей слова состояния доступны в переменных брокера `sys.alarmN` и `sys.statusN`, где `N` – позиционный номер модуля. Для модулей с отрицательными позиционными номерами используются переменные `sys.alarm_N` и `sys.status_N`. Диагностическая информация головного модуля ПЛК дополнительно доступна в переменных `sys.alarm` и `sys.status`, независимо позиционного номера модуля, исполняющего в данный момент роль головного.

Переменные `sys.alarmN` содержат биты ошибок. Назначение битов в переменных `sys.alarmN` одинаковое для всех модулей ПЛК Logic Box. Головной модуль дополнительно формирует переменную `sys.allalarm`, содержащую побитовое ИЛИ текущего значения переменных `sys.alarmN` всех модулей; при нормальной работе ПЛК все переменные `sys.alarmN` и переменная `sys.allalarm` имеют значение «0».

Переменные `sys.statusN` содержат биты состояния типов `E` и `I`. Биты типа `E` конкретизируют причины установки битов ошибок `sys.alarmN`; эти биты имеют значение «0» при нормальной работе ПЛК. Биты типа `I` носят информационный характер и могут иметь значения «0» или «1» при нормальной работе ПЛК. Назначение битов `sys.statusN` специфично для каждого типа модуля.

Переменные состояния доступны для считывания по протоколу `prc21_modbus`, а также могут передаваться в прикладные программы ПЛК и другие подсистемы средствами конфигурации. Например, можно настроить доступ к переменным состояния по протоколу `Modbus`:

```
modbus_server:
  inreg101..120: {var_out: sys.status1..20}
  inreg201..220: {var_out: sys.alarm1..20}
  inreg299: {var_out: sys.allalarm}
```

Назначение битов переменных состояния для каждого типа модулей ПЛК Logic Box описано в таблицах 2–13.

Таблица 2 – Назначение битов `sys.alarmN` всех модулей

Номер бита	Обозн.	Тип	Описание
15	MA	E	«Module absent»: модуль не присылал диагностическую информацию головному модулю в течение последних 8 с (отсутствует или неисправен)
14	UE	E	«Unclassified error»: любая ошибка, не имеющая отдельного бита в <code>sys.alarmN</code> . Детализация в <code>sys.statusN</code> и/или <code>lblog</code>
13			резерв
12	MC	E	«Modbus client»: ошибка клиента Modbus: нет ответа сервера, ошибочный ответ сервера (exception), ошибка CRC
11	VB	E	«Battery voltage»: неисправность автономного источника питания часов реального времени
10	RU	E	«PLC run mode»: выполнение прикладных программ ПЛК остановлено (переменная <code>sys.run</code> имеет значение «0», установленное по ошибке выполнения прикладной программы или по команде оператора)
9	IP	E	«IP address»: модуль не имеет IP-адреса (ни статического ни назначенного по протоколу DHCP)
8	PA	E	«Power absent»: модуль является инжектором питания шины ПЛК (LB241BC, LB241CPU, LB340CPU, LB340EXT, LB340PWR), и в последние 3с напряжение на разъеме внешнего питания модуля не приходило в норму ($24V \pm 20\%$)
7	PW	E	«Power error»: модуль является инжектором питания шины ПЛК, и за последние 3с были замечены отклонения напряжения на разъеме внешнего питания модуля от нормы ($24V \pm 20\%$)
6	PI	E	«I/O power» в последние 3с замечены отсутствие или отклонения от нормы дополнительного напряжения питания модуля ввода-вывода (на модулях LB241DO16, LB340DO32, LB241MG)
5	AC	E	«Alternate current»: замечено присутствие переменного напряжения 220В 50Гц на порту ввода-вывода или входе дополнительного напряжения питания модуля ввода-вывода (на модулях LB241DO16, LB340DO32, LB241DI16, LB340DI32). Порт переведен в состояние защиты

см. далее

Продолжение таблицы 2

Номер бита	Обозн.	Тип	Описание
4	VI	E	«Internal voltage»: замечено отклонение от нормы одного из внутренних напряжений питания модуля (самодиагностика)
3	CH	E	«I/O channel»: один или более канал ввода-вывода находится в состоянии ошибки; детализация в <code>sys.statusN</code> и/или <code>lblog</code>
2	NC	E	«Need configuration»: в модуль не загружена конфигурация
1	BF	E	«Bus failure»: долговременная ошибка шины ПЛК, передача данных невозможна
0	BE	E	«Bus error»: за последние 3 с были замечены сбои в работе шины ПЛК, например ошибки CRC

Таблица 3 – Назначение битов `sys.statusN` модуля LB241BC

Номер бита	Обозн.	Тип	Описание
15	BM	I	модуль работает в роли головного модуля шины
14			резерв
13			резерв
12			резерв
11			резерв
10			резерв
9			резерв
8			резерв
7			резерв
6			резерв
5			резерв
4	S0	I	порт COM работает в режиме ведомого (сервера)
3			резерв
2			резерв
1			резерв
0	M0	I	порт COM работает в режиме ведущего (клиента)

Таблица 4 – Назначение битов `sys.statusN` модуля LB241CPU

Номер бита	Обозн.	Тип	Описание
15	BM	I	модуль работает в роли головного модуля шины
14			резерв
13			резерв
12			резерв
11			резерв
10			резерв
9			резерв
8			резерв
7			резерв
6			резерв
5			резерв
4	S0	I	порт COM работает в режиме ведомого (сервера)
3			резерв
2			резерв
1			резерв
0	M0	I	порт COM работает в режиме ведущего (клиента)

Таблица 5 – Назначение битов `sys.statusN` модуля LB241DI16

Номер бита	Обозн.	Тип	Описание
15	C15	E	ошибка на канале 15
14	C14	E	ошибка на канале 14
13	C13	E	ошибка на канале 13
12	C12	E	ошибка на канале 12
11	C11	E	ошибка на канале 11
10	C10	E	ошибка на канале 10
9	C9	E	ошибка на канале 9
8	C8	E	ошибка на канале 8
7	C7	E	ошибка на канале 7
6	C6	E	ошибка на канале 6
5	C5	E	ошибка на канале 5
4	C4	E	ошибка на канале 4
3	C3	E	ошибка на канале 3
2	C2	E	ошибка на канале 2
1	C1	E	ошибка на канале 1
0	C0	E	ошибка на канале 0

Таблица 6 – Назначение битов `sys.statusN` модуля LB241DO16

Номер бита	Обозн.	Тип	Описание
15	C15	E	ошибка на канале 15
14	C14	E	ошибка на канале 14
13	C13	E	ошибка на канале 13
12	C12	E	ошибка на канале 12
11	C11	E	ошибка на канале 11
10	C10	E	ошибка на канале 10
9	C9	E	ошибка на канале 9
8	C8	E	ошибка на канале 8
7	C7	E	ошибка на канале 7
6	C6	E	ошибка на канале 6
5	C5	E	ошибка на канале 5
4	C4	E	ошибка на канале 4
3	C3	E	ошибка на канале 3
2	C2	E	ошибка на канале 2
1	C1	E	ошибка на канале 1
0	C0	E	ошибка на канале 0

Таблица 7 – Назначение битов `sys.statusN` модуля LB241AI4

Номер бита	Обозн.	Тип	Описание
24	TR	E	ошибки датчиков температуры модуля (Tref)
23			резерв
22			резерв
21			резерв
20	HP	E	перегрузка преобразователя питания каналов
19	F3	E	неклассифицированная ошибка на канале 3
18	F2	E	неклассифицированная ошибка на канале 2
17	F1	E	неклассифицированная ошибка на канале 1
16	F0	E	неклассифицированная ошибка на канале 0
15	B3	E	обрыв на канале 3
14	B2	E	обрыв на канале 2
13	B1	E	обрыв на канале 1
12	B0	E	обрыв на канале 0
11	S3	E	КЗ на канале 3
10	S2	E	КЗ на канале 2
9	S1	E	КЗ на канале 1
8	S0	E	КЗ на канале 0
7	P3	E	сработала защита на канале 3
6	P2	E	сработала защита на канале 2
5	P1	E	сработала защита на канале 1
4	P0	E	сработала защита на канале 0
3	D3	I	канал 3 не описан в конфигурации
2	D2	I	канал 2 не описан в конфигурации
1	D1	I	канал 1 не описан в конфигурации
0	D0	I	канал 0 не описан в конфигурации

Таблица 8 – Назначение битов `sys.statusN` модуля LB241AO4

Номер бита	Обозн.	Тип	Описание
21	V5	E	отклонение внутреннего напряжения 5В от нормы
20	V10	E	отклонение внутреннего напряжения 10В от нормы
19			резерв
18			резерв
17			резерв
16	HP	E	перегрузка преобразователя питания каналов
15	B3	E	обрыв на канале 3
14	B2	E	обрыв на канале 2
13	B1	E	обрыв на канале 1
12	B0	E	обрыв на канале 0
11	S3	E	КЗ на канале 3
10	S2	E	КЗ на канале 2
9	S1	E	КЗ на канале 1
8	S0	E	КЗ на канале 0
7	F3	E	неклассифицированная ошибка на канале 3
6	F2	E	неклассифицированная ошибка на канале 2
5	F1	E	неклассифицированная ошибка на канале 1
4	F0	E	неклассифицированная ошибка на канале 0
3	D3	I	канал 3 не описан в конфигурации
2	D2	I	канал 2 не описан в конфигурации
1	D1	I	канал 1 не описан в конфигурации
0	D0	I	канал 0 не описан в конфигурации

Таблица 9 – Назначение битов `sys.statusN` модуля LB340CPU

Номер бита	Обозн.	Тип	Описание
15	BM	I	модуль работает в роли головного модуля шины
14	Vi	E	неисправность ионистора
13	Vb	E	батарея отсутствует или разряжена
12	PВ	E	отклонение напряжения 24В на шине от нормы
11	V5	E	отклонение внутреннего напряжения 5В от нормы
10			резерв
9			резерв
8			резерв
7			резерв
6			резерв
5			резерв
4	S0	I	порт COM работает в режиме ведомого (сервера)
3			резерв
2			резерв
1			резерв
0	M0	I	порт COM работает в режиме ведущего (клиента)

Таблица 10 – Назначение битов `sys.statusN` модуля LB340DI32

Номер бита	Обозн.	Тип	Описание
33	HP1	E	перегрузка преобразователя питания каналов 16..31
32	HP0	E	перегрузка преобразователя питания каналов 0..15
31	C31	E	ошибка на канале 31
30	C30	E	ошибка на канале 30
...		E	
15	C15	E	ошибка на канале 15
14	C14	E	ошибка на канале 14
13	C13	E	ошибка на канале 13
12	C12	E	ошибка на канале 12
11	C11	E	ошибка на канале 11
10	C10	E	ошибка на канале 10
9	C9	E	ошибка на канале 9
8	C8	E	ошибка на канале 8
7	C7	E	ошибка на канале 7
6	C6	E	ошибка на канале 6
5	C5	E	ошибка на канале 5
4	C4	E	ошибка на канале 4
3	C3	E	ошибка на канале 3
2	C2	E	ошибка на канале 2
1	C1	E	ошибка на канале 1
0	C0	E	ошибка на канале 0

Таблица 11 – Назначение битов `sys.statusN` модуля LB340DO32

Номер бита	Обозн.	Тип	Описание
35	P1	E	внешнее питание +24В каналов 16..31 отсутствует
34	P0	E	внешнее питание +24В каналов 0..15 отсутствует
33	HP1	E	перегрузка преобразователя питания каналов 16..31
32	HP0	E	перегрузка преобразователя питания каналов 0..15
31	C31	E	ошибка на канале 31
30	C30	E	ошибка на канале 30
...		E	
15	C15	E	ошибка на канале 15
14	C14	E	ошибка на канале 14
13	C13	E	ошибка на канале 13
12	C12	E	ошибка на канале 12
11	C11	E	ошибка на канале 11
10	C10	E	ошибка на канале 10
9	C9	E	ошибка на канале 9
8	C8	E	ошибка на канале 8
7	C7	E	ошибка на канале 7
6	C6	E	ошибка на канале 6
5	C5	E	ошибка на канале 5
4	C4	E	ошибка на канале 4
3	C3	E	ошибка на канале 3
2	C2	E	ошибка на канале 2
1	C1	E	ошибка на канале 1
0	C0	E	ошибка на канале 0

Таблица 12 – Назначение битов `sys.statusN` модуля LB340AI4

Номер бита	Обозн.	Тип	Описание
24	TR	E	ошибки датчиков температуры модуля (Tref)
23	HP3	E	перегрузка преобразователя питания канала 3
22	HP2	E	перегрузка преобразователя питания канала 2
21	HP1	E	перегрузка преобразователя питания канала 1
20	HP0	E	перегрузка преобразователя питания канала 0
19	F3	E	неклассифицированная ошибка на канале 3
18	F2	E	неклассифицированная ошибка на канале 2
17	F1	E	неклассифицированная ошибка на канале 1
16	F0	E	неклассифицированная ошибка на канале 0
15	B3	E	обрыв на канале 3
14	B2	E	обрыв на канале 2
13	B1	E	обрыв на канале 1
12	B0	E	обрыв на канале 0
11	S3	E	K3 на канале 3
10	S2	E	K3 на канале 2
9	S1	E	K3 на канале 1
8	S0	E	K3 на канале 0
7	P3	E	сработала защита на канале 3
6	P2	E	сработала защита на канале 2
5	P1	E	сработала защита на канале 1
4	P0	E	сработала защита на канале 0
3	D3	I	канал 3 не описан в конфигурации
2	D2	I	канал 2 не описан в конфигурации
1	D1	I	канал 1 не описан в конфигурации
0	D0	I	канал 0 не описан в конфигурации

Таблица 13 – Назначение битов sys.statusN модуля LB340AO4

Номер бита	Обозн.	Тип	Описание
21	V5	E	отклонение внутреннего напряжения 5В от нормы
20	V88	E	отклонение внутреннего напряжения 8.8В от нормы
19	HP3	E	перегрузка преобразователя питания канала 3
18	HP2	E	перегрузка преобразователя питания канала 2
17	HP1	E	перегрузка преобразователя питания канала 1
16	HP0	E	перегрузка преобразователя питания канала 0
15	B3	E	обрыв на канале 3
14	B2	E	обрыв на канале 2
13	B1	E	обрыв на канале 1
12	B0	E	обрыв на канале 0
11	S3	E	КЗ на канале 3
10	S2	E	КЗ на канале 2
9	S1	E	КЗ на канале 1
8	S0	E	КЗ на канале 0
7	F3	E	неклассифицированная ошибка на канале 3
6	F2	E	неклассифицированная ошибка на канале 2
5	F1	E	неклассифицированная ошибка на канале 1
4	F0	E	неклассифицированная ошибка на канале 0
3	D3	I	канал 3 не описан в конфигурации
2	D2	I	канал 2 не описан в конфигурации
1	D1	I	канал 1 не описан в конфигурации
0	D0	I	канал 0 не описан в конфигурации

7. СООБЩЕНИЯ

Программа передает ведущему модулю информацию об ошибках исполнения команд и других событиях в текстовом виде в пакетах LOG.

7.1. Сообщение «OK conf NNNNNNNN»

Сообщение об успешном разборе конфигурации. Подстрока NNNNNNNN представляет собой первые 8 символов хэш-суммы SHA256 от конфигурации в формате JSON. Эта подстрока предназначена для идентификации версии конфигурации в системном журнале.

7.2. Сообщение «expected var name after var_out»

Сообщение об ошибке разбора: после ключевого слова var_out должно следовать имя переменной.

7.3. Сообщение «out of memory»

Сообщение о нехватке оперативной памяти. Не должно возникать при нормальной работе программы. Может возникнуть при чрезмерной длине пакета конфигурации, или в из-за ошибки в программе.

7.4. Сообщение «unexpected item»

Сообщение об ошибке разбора: обнаружено недопустимое имя параметра. Допустимые имена параметров перечислены в разделе 5.

7.5. Сообщение «object expected»

Сообщение об ошибке разбора: нарушение структуры конфигурации, вместо подраздела («object») обнаружен другой элемент, например строка.

7.6. Сообщение «expected "module: DO"»

Сообщение об ошибке разбора: в конфигурации отсутствует параметр `module`: или значение параметра `module`: не совпадает с типом модуля («D0»).

7.7. Сообщение «bad out-channel number, expected out0 ... out63»

Сообщение об ошибке разбора: в параметре `out $n$` указан несуществующий номер выхода n .

7.8. Сообщение «duplicate out-channel»

Сообщение об ошибке разбора: в конфигурации обнаружено более одного подраздела `out $n$` с одним и тем же n .

7.9. Сообщение «expected `module', 'init' or `outN'»

Сообщение об ошибке разбора: обнаружено недопустимое имя параметра на верхнем уровне. Допустимые имена перечислены в разделе 5.

7.10. Сообщение «cJSON_Parse failed»

Сообщение об ошибке разбора: нарушение формата JSON. Не должно возникать при использовании штатных средств трансляции и загрузки конфигурации.

ПЕРЕЧЕНЬ ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.504-79. ЕСПД. Руководство программиста. Требования к содержанию и оформлению. — М. : Стандартиформ, 1979.
2. The Official YAML Web Site. — 2004. — Access mode: <https://yaml.org>.
3. MATIEC - IEC 61131-3 compiler. — 2012. — Access mode: <https://github.com/beremiz/matiec>.
4. ESP-IDF Programming Guide. — <https://docs.espressif.com/projects/esp-idf/en/latest/>. — [Интернет-ресурс, дата доступа 29.10.2023].
5. Eclipse 4diacTM) - Open Source for Industrial Automation Systems. — 2004. — Access mode: <https://eclipse.dev/4diac/>.
6. ГОСТ Р МЭК 61131-3-2016. Контроллеры программируемые. Часть 3. Языки программирования. — 2004.
7. MODBUS Application Protocol Specification V1.1b. — https://www.modbus.org/docs/Modbus_Application_Protocol_V1_1b.pdf. — 2006. — [Интернет-ресурс, дата доступа 29.10.2023].
8. ГОСТ Р МЭК 60870-5-104-2004. Устройства и системы телемеханики. Часть 5. Протоколы передачи. Раздел 104. Доступ к сети для ГОСТ Р МЭК 870-5-101 с использованием стандартных транспортных профилей. — 2004.
9. Репозиторий НПЦ ИТ. — 2025. — Access mode: <https://git.it-npc.ru>.
10. regex(7)—Linux manual page. — 2024. — Access mode: <https://www.man7.org/linux/man-pages/man7/regex.7.html>.
11. ГОСТ Р МЭК 870-5-5. Устройства и системы телемеханики. Часть 5. Протоколы передачи. Раздел 5. Основные прикладные функции. — 1996.
12. ГОСТ Р МЭК 60870-5-101-2006. Устройства и системы телемеханики. Часть 5. Протоколы передачи. Раздел 101. Обобщающий стандарт по основным функциям телемеханики. — 2006.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ

IEC-104 – протокол управления ГОСТ Р МЭК 60870-5-104

SDK – система сборки ПО (software development kit)

КЗ – короткое замыкание

ОС – операционная система

ПО – программное обеспечение

ПЛК – программируемый логический контроллер

СнК – система на кристалле (SoC, system on chip)

УСО – устройство связи с объектом

[illegible][illegible]